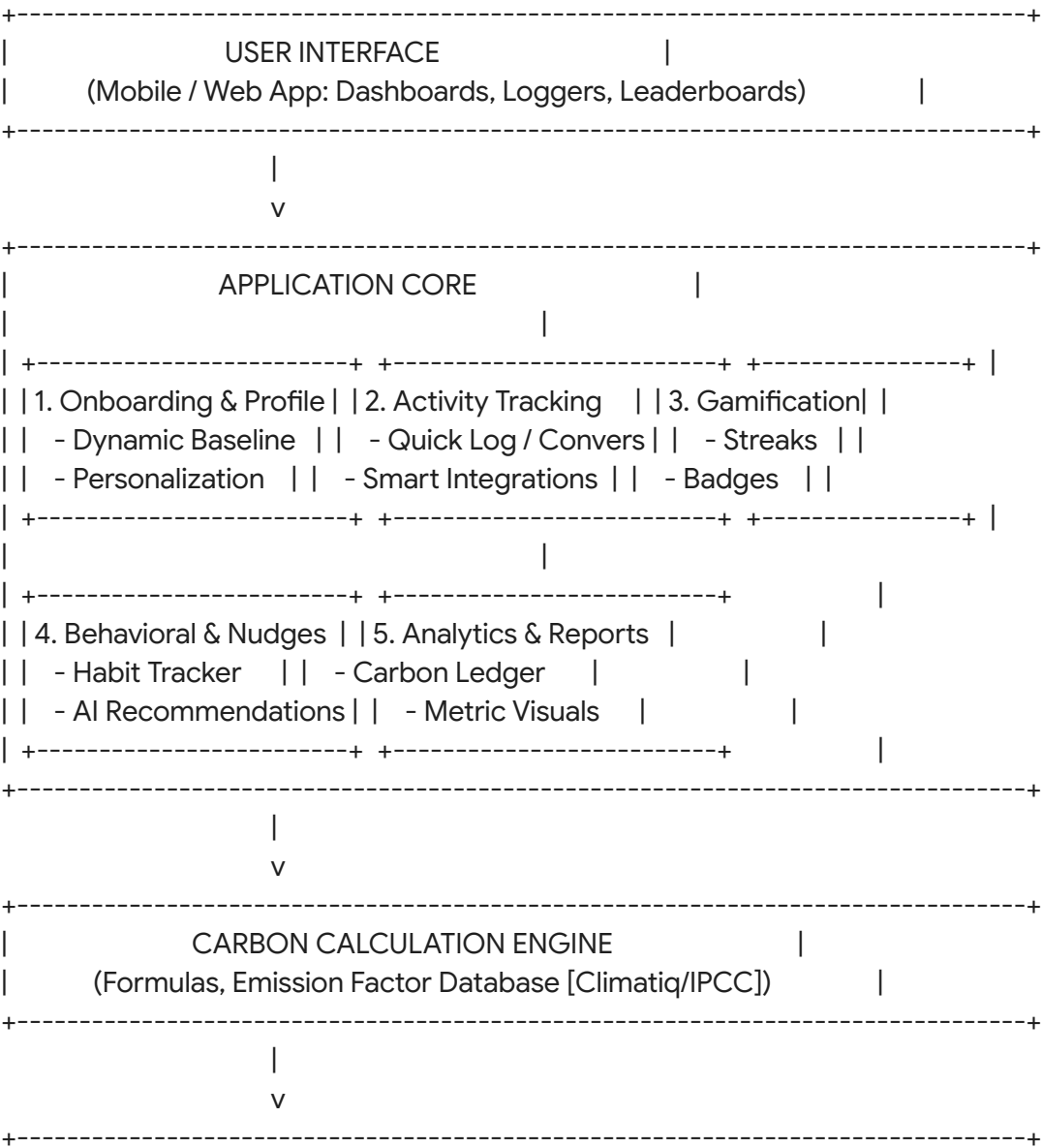


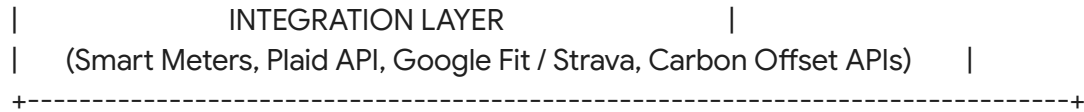
Carbon Footprint Awareness Platform: Feature Architecture Spec

This document details the functional specifications, logical modules, data architectures, and calculation frameworks required to build a scalable, high-retention Carbon Footprint Awareness Platform.

1. High-Level System Architecture

The application is structured into five core functional modules supported by a centralized Calculation Engine and an Integration Layer.





2. Module Breakdown & Feature Specifications

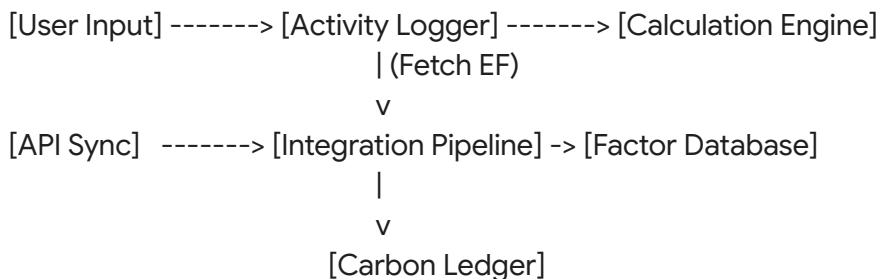
Module 1: Onboarding & Dynamic Baseline Assessment

Responsible for establishing a frictionless, accurate starting footprint without causing questionnaire fatigue.

- Frictionless Baseline Calculator**
 - Description:* An interactive, multi-step onboarding flow (approx. 5–8 questions max) using sliding scales and icons.
 - Inputs:* Location (postal/zip code), diet type (Vegan, Vegetarian, Flexitarian, High Meat), housing type (Apartment, House), transport mix (Car, Transit, Flight hours).
 - Logic/Processing:* Map answers to regional averages using dynamic carbon models rather than demanding exact values.
 - Output:* Initial annual carbon baseline in metric tons of CO_2 equivalent ($tCO_2e/year$).
- Target Setter (Goal Customization)**
 - Description:* Allows users to choose an emission reduction path.
 - Modes:* *Eco-Warrior* (50% reduction in line with IPCC Paris Goals), *Pace-Setter* (20% reduction), or *Custom*.
 - Output:* Dynamic generation of daily and weekly carbon budgets ($kgCO_2e$).

Module 2: Multi-Modal Activity Tracking Engine

Processes user activities into standardized emission entries. It supports three logging modalities: manual, quick-select, and automated sync.



- Quick-Log Interface**
 - Description:* A 1-tap logging interface for recurring daily actions (e.g., "Commuting by

subway", "Had a plant-based meal").

- *Inputs*: Activity category templates.
- *Processing*: Instantly maps action to predefined baseline values (e.g., Subway Commute $\approx 0.03 \text{ kgCO}_2\text{e/km}$).

- **Granular Manual Loggers**

- *Transport Log*: Input vehicle type, fuel type (petrol, EV, diesel), and distance.
- *Diet Log*: Meal-level logging categorized by primary protein sources.
- *Energy Log*: Input of utility bills (electricity in kWh , gas in m^3).

- **Automated Sync Pipelines (Advanced MVP)**

- *Strava / Google Fit Integration*: Auto-logs walking, running, or cycling activities, marking them as 0 -emission travel alternatives.
- *Smart Meter Integrations (UtilityAPI)*: Automates household energy tracking via live utility grid updates.

Module 3: Carbon Analytics & Ledger

The central data repository and visualization module, designed to make invisible emissions visible and contextual.

- **The Carbon Ledger**

- *Description*: A double-entry style database table tracking positive emissions (activities) and negative emissions (offsets/reductions) measured in $kgCO_2e$.

- **Visual Analytics Dashboard**

- *Feature Breakdown*:
 - *Category Distribution*: A dynamic donut chart illustrating the breakdown (Housing vs. Transport vs. Diet vs. Consumption).
 - *Temporal Trends*: Line chart showing weekly/monthly progress relative to the carbon budget line.

- **Eco-Equivalence Translator**

- *Description*: Micro-copy engine translating abstract metric tons of CO_2 into tangible real-world analogies.
- *Formula Translation Examples*:
 - $1 \text{ kgCO}_2\text{e} \approx 4 \text{ smartphone charges}$.
 - $22 \text{ kgCO}_2\text{e} \approx \text{Abortion capacity of 1 mature tree in a year}$.

Module 4: Behavioral Nudges & Habit Tracker

Drives behavioral modification by reinforcing atomic habits rather than merely presenting data.

- **Habit Builder & Checklists**

- *Description*: A customizable habit dashboard where users commit to micro-actions.

- *Features:* Daily checklists, smart reminders (push notifications), and a "Habit Streak" tracker.
- **Predictive Suggestion Engine (AI Nudges)**
 - *Description:* Scans the user's weekly ledger to detect anomalies and trigger targeted advice.
 - *Rule-based Logic Example:*

IF (Transport $CO_2 > 50\%$ of weekly total) AND (Distance $< 5\text{km}$) $\rightarrow$ TRIGGER: "T

Module 5: Gamification & Collective Impact

Ensures user retention by turning tracking into a collaborative, rewarding experience.

- **Social Challenges & Peer-to-Peer Leagues**
 - *Description:* Custom lobbies where user-groups compete in structured, time-bound challenges (e.g., "7-Day No-Red-Meat Challenge").
 - *Mechanics:* Leaderboard sorted by total carbon saved or habit compliance percentage.
- **Leveling System & Eco-Credentials**
 - *Description:* Users earn XP (Experience Points) and level up.
 - *Levels:* Sprout (Level 1) $\rightarrow$ Sapling $\rightarrow$ Oak $\rightarrow$ Redwood (Level 10).
- **Green Reward Marketplace**
 - *Description:* Users exchange accumulated points earned through streak completions for green rewards.
 - *Partnership Integrations:* Discount codes on ethical/eco-friendly brands, or direct contributions to verified carbon capture (e.g., planting trees via Eden Reforestation Projects).

3. The Carbon Calculation Engine

Every logged activity is processed using the standard global Greenhouse Gas (GHG) Protocol calculation framework:

$$\text{Carbon Footprint (kgCO}_2\text{e)} = \text{Activity Data (AD)} \times \text{Emission Factor (EF)}$$

Emission Calculations by Scope

1. Transport Emissions

$$E_{\text{transport}} = d \times EF_{\text{vehicle_type}}$$

•

Where:

- d = Distance traveled (km).
- $EF_{\text{vehicle_type}}$ = Emission factor for the vehicle model/fuel type (e.g., Gasoline Sedan: $0.17 kgCO_2e/km$; Electric Vehicle: Grid intensity dependent).

2. Food & Diet Emissions

$$E_{\text{diet}} = \sum_i (w_i \times EF_i)$$

•

Where:

- w_i = Mass of food item i in kg .
- EF_i = Carbon footprint of food class (e.g., Beef: $27.0 kgCO_2e/kg$, Tofu: $2.0 kgCO_2e/kg$).

3. Home Utilities (Electricity)

$$E_{\text{electricity}} = C_{\text{kWh}} \times EF_{\text{local_grid}}$$

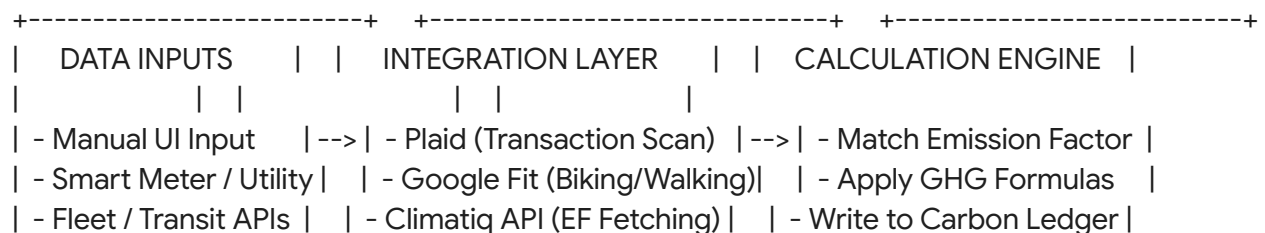
•

Where:

- C_{kWh} = Power consumption in kilowatt-hours.
- $EF_{\text{local_grid}}$ = Regional grid emission factor ($kgCO_2e/kWh$), representing the mix of coal, gas, and renewables in the local energy grid.

4. Integration & Data Flow Architecture

The platform relies on third-party APIs to automate tracking and fetch reliable emission factors.



Core API Integrations

1. **Climatiq API:** For pulling real-time, localized GHG Protocol emission factors across transport, travel, energy, and supply chain logistics.
2. **Plaid API (Optional for Carbon/Fintech MVP):** Matches transactional data (e.g., fuel station charge, grocery store checkout) with consumption categories to estimate carbon values using transaction-to-CO2 algorithms.
3. **Electricity Maps API:** Retrieves real-time grid carbon intensity factors per country/state for precise home-energy calculations.
4. **Patch / Cloverly API:** Integration with verified programmatic carbon offset providers to facilitate micro-offsets natively inside the marketplace module.

5. Technical Database Schema (Prerequisite for MVP)

A relational database structure designed to log, aggregate, and query carbon ledger data efficiently.

-- Users and Profile Configurations

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  baseline_co2 DECIMAL(10,2), -- Target tCO2e per year  
  weekly_budget DECIMAL(10,2) -- Target kgCO2e per week  
);
```

-- Core Ledger tracking all CO2 records (In and Out)

```
CREATE TABLE carbon_ledger (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  category VARCHAR(50) NOT NULL, -- 'transport', 'diet', 'utilities', 'offset'  
  sub_category VARCHAR(100), -- 'flight', 'beef', 'solar_offset'  
  quantity DECIMAL(10,2) NOT NULL, -- raw unit (e.g., 15 km, 2 kWh)  
  unit VARCHAR(20) NOT NULL, -- 'km', 'kg', 'kWh'  
  emissions_saved_or_spent DECIMAL(10,4) NOT NULL, -- calculated footprint (kgCO2e)  
  logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Active User Challenges

```
CREATE TABLE user_challenges (  
  id UUID PRIMARY KEY,
```

```
user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
challenge_name VARCHAR(255) NOT NULL,  
start_date DATE NOT NULL,  
end_date DATE NOT NULL,  
status VARCHAR(20) DEFAULT 'active' -- 'active', 'completed', 'failed'  
);
```